

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD PIPELINE DESIGN EXERCISE*CI/CD Pipeline Design Document*

Course Code	CSA10	Course Title	Software Engineering
CO Assessed	CO4	Assessment Tool	Tool 2 – CI/CD Pipeline Design
Problem No.	13		
Student Name	S Suvikumar	Reg. No.	192424262
Date	19/08/2026		

CO4 Statement

Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics.

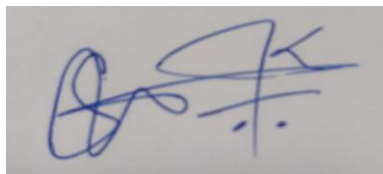
Assigned Problem Statement**Intelligent E-Governance Citizen Service Request Management System.**

Roads frequently develop cracks, potholes, and surface deterioration due to traffic load and environmental conditions. This document designs a CI/CD (Continuous Integration and Continuous Deployment) pipeline that automates the build, testing, quality analysis, packaging, and deployment of the AI-based road-damage detection and maintenance-planning platform, across development, staging, and production environments, with built-in security scanning, notifications, and rollback mechanisms.

Code of Conduct

I **S SUVIKUMAR / 192424262** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Student Signature



1. Requirement & CI/CD Needs Analysis

1.1 Project Characteristics Driving CI/CD Design

The AI-Based Road Surface Damage Detection and Maintenance Planning System is composed of multiple independently deployable microservices (image ingestion, AI damage detection, classification, severity estimation, GIS mapping, scheduling, reporting, notification) built on a hybrid microservices and event-driven architecture. This composition directly shapes the CI/CD requirements: each service must be built, tested, and deployed independently; the AI models require additional validation beyond standard unit tests; and releases must be safely rolled out to a live public-safety system without downtime.

1.2 Identified CI/CD Requirements

- **Independent, per-service pipelines:** each microservice (detection, classification, GIS, scheduling, etc.) needs its own build/test/deploy pipeline so that a change to one service does not force a redeploy of the entire platform.
- **Automated multi-layer testing:** unit, integration, and AI-model accuracy/regression tests must run automatically on every commit before code can be merged.
- **Code quality and security gates:** static analysis, dependency-vulnerability scanning, and container-image scanning must block low-quality or insecure builds from progressing.
- **Environment promotion:** changes must be automatically promoted through Development → Staging → Production, with automated tests at each stage and a manual approval gate before production.
- **Safe, zero-downtime deployment:** since the system feeds real-time defect alerts to municipal authorities, production deployments must use blue-green or canary strategies with automated rollback.
- **Model versioning and traceability:** AI model artifacts (weights, training data version) must be versioned alongside code so that any detection result can be traced back to a specific model build.
- **Observability and notification:** the pipeline must integrate monitoring/alerting and notify the development and operations teams of build, test, and deployment status in real time.

2. Pipeline Architecture, Workflow, and Diagram

The proposed pipeline follows eleven stages, grouped into four phases: Continuous Integration (commit through packaging), Continuous Deployment to lower environments (development and staging), a controlled promotion gate, and Continuous Deployment to production with monitoring and rollback. The full architecture is shown below.

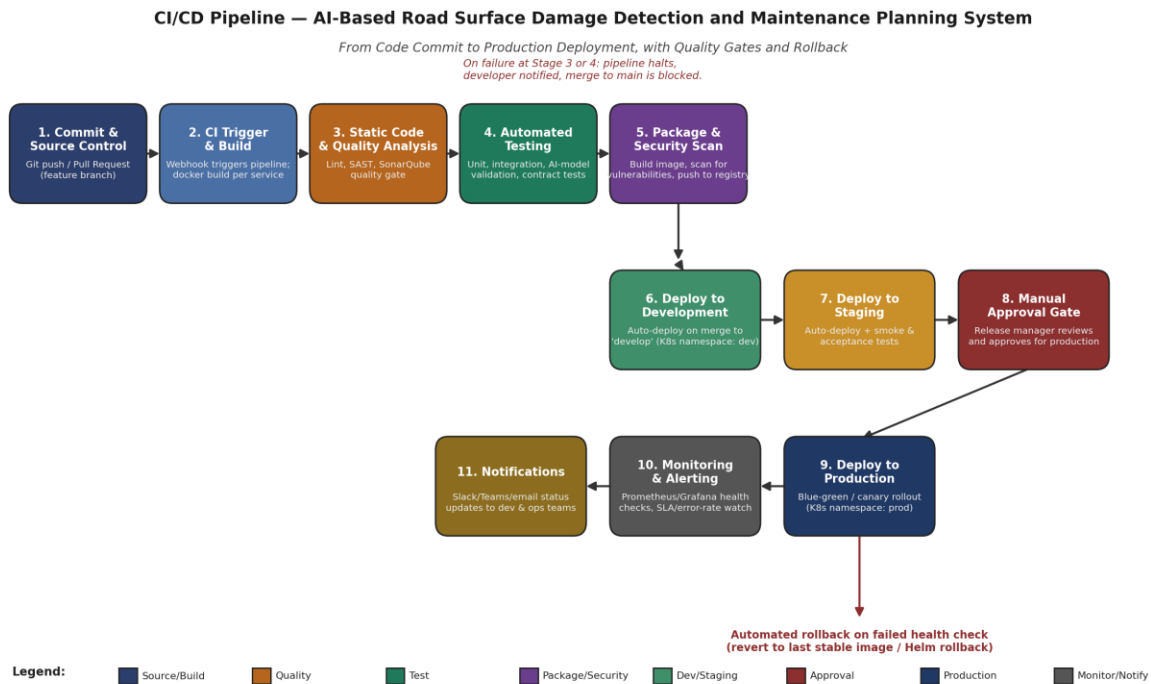


Figure 1: End-to-end CI/CD pipeline for the AI-Based Road Surface Damage Detection and Maintenance Planning System

2.1 Workflow Explanation — Commit to Deployment

- **Commit & Source Control:** a developer pushes code to a feature branch in Git (GitHub/GitLab) and opens a pull request; branch-protection rules require the pipeline to pass before merge.
- **CI Trigger & Build:** a webhook automatically triggers the pipeline; each affected microservice is containerized via a multi-stage Docker build, so only changed services are rebuilt.
- **Static Code & Quality Analysis:** linting and Static Application Security Testing (SAST) run against the codebase, and a SonarQube quality gate checks maintainability, code smells, and duplication; failures halt the pipeline and notify the developer.
- **Automated Testing:** unit tests, integration tests, and AI-model validation tests (accuracy/precision-recall against a held-out labeled dataset) run in parallel; contract tests confirm inter-service API compatibility.

- **Package & Security Scan:** passing builds are packaged into versioned Docker images, scanned for known vulnerabilities (container and dependency scanning), and pushed to a private container registry with semantic-version and Git-commit tags.
- **Deploy to Development:** the pipeline auto-deploys the new images to the development Kubernetes namespace for early integration testing by the team.
- **Deploy to Staging:** on success, the same artifacts are promoted (not rebuilt) to a staging environment that mirrors production, where automated smoke and acceptance tests run against real GIS/map integrations.
- **Manual Approval Gate:** a release manager reviews staging test results and explicitly approves promotion to production, adding a human safety check before a public-safety system is updated.
- **Deploy to Production:** approved artifacts are rolled out using a blue-green or canary strategy so that only a small percentage of traffic (or a parallel environment) sees the new version initially.
- **Monitoring & Alerting:** Prometheus/Grafana continuously track error rates, latency, and AI-service health after deployment; predefined SLA thresholds trigger automated rollback if breached.
- **Notifications:** Slack/Teams/email notifications are sent at every key stage (build failure, test failure, successful deployment, rollback) to keep both developers and operations informed in real time.

3. Tools & Technology Selection

Tools were selected to fit the heterogeneous, microservices-based technology stack described in the system's architecture, favoring Kubernetes-native, widely supported, and cost-effective options.

Pipeline Stage	Selected Tool(s)	Justification
Source Control & Trigger	GitHub + GitHub Actions (webhooks)	Widely adopted, tightly integrates version control with CI triggers, and supports branch-protection rules that enforce pipeline checks before merge.
Build & Containerization	Docker + Docker Buildx	Enables consistent, reproducible builds across the heterogeneous microservices (Python AI services, Node/Java core services) and produces portable images for Kubernetes.
Static Code Analysis	SonarQube + ESLint/PyLint	SonarQube provides a configurable quality gate (coverage, duplication, code smells) across multiple languages used by different microservices; linters catch style/basic errors early and cheaply.

Pipeline Stage	Selected Tool(s)	Justification
Security Scanning (SAST/Deps)	Snyk / OWASP Dependency-Check	Detects known vulnerabilities in application code and third-party dependencies before they reach a running environment, critical given the system stores location and personal contact data.
Automated Testing	PyTest (AI services), Jest/JUnit (core services), Postman/Newman (API contract tests)	Matches each service's native language/tooling, minimizing friction for developers, while Postman/Newman verifies REST contracts between independently deployed microservices.
Container Security Scan	Trivy	Lightweight, fast vulnerability scanner for container images and IaC files; integrates easily as a pipeline step before pushing to the registry.
Artifact/Container Registry	Amazon ECR / Docker Hub (private)	Provides versioned, access-controlled storage for built images, supporting rollback to any previously pushed, known-good version.
Orchestration & Deployment	Kubernetes + Helm	Namespaces cleanly separate dev/staging/prod; Helm charts template each microservice's deployment and make blue-green/canary rollouts and one-command rollbacks straightforward.
CD Automation	Argo CD (GitOps)	Continuously reconciles the cluster state with a Git-stored desired state, giving an auditable deployment history and simple, declarative rollback.
Monitoring & Alerting	Prometheus + Grafana	Industry-standard, Kubernetes-native metrics collection and dashboards; supports the SLA-based alert rules that trigger automated rollback.
Notifications	Slack / Microsoft Teams webhooks	Delivers real-time build/test/deploy status to the whole team with minimal setup, integrated directly into the CI/CD tool's notification steps.

4. Automated Testing Strategy

4.1 Test Layers Integrated into the Pipeline

- **Unit Tests:** run on every commit for each microservice (e.g., severity-score calculation logic, GIS coordinate validation); must maintain a minimum 80% code-coverage gate to pass the pipeline.
- **Integration Tests:** verify interactions between services and their datastores (e.g., Image Ingestion Service writing to the object store and publishing to the event bus) using ephemeral test containers.
- **AI Model Validation Tests:** run the current model against a fixed, version-controlled held-out dataset and assert accuracy, precision, and recall stay within an approved tolerance of the previous release, preventing silent model-quality regressions.
- **Contract/API Tests:** Postman/Newman collections validate that each microservice's REST API still matches its published contract, protecting independently deployed services from breaking one another.
- **Security Tests:** SAST and dependency/container scans (Snyk, Trivy) run automatically in the pipeline and fail the build on high/critical vulnerabilities.
- **Smoke & Acceptance Tests:** run automatically immediately after deployment to staging, exercising key user journeys (image upload → defect detected → appears on GIS map) end-to-end.
- **Post-Deployment Health Checks:** Kubernetes readiness/liveness probes plus Prometheus alert rules continuously validate production health and feed the automated-rollback decision.

4.2 Quality Gates

A build is blocked from progressing to the next stage unless: (a) all unit and integration tests pass, (b) code coverage remains at or above the configured threshold, (c) the SonarQube quality gate passes, and (d) no high/critical vulnerabilities are found in code, dependency, or container scans. This ensures only thoroughly verified, secure builds reach staging and production.

5. Deployment Strategy

The pipeline promotes a single, immutable build artifact through three environments rather than rebuilding at each stage, guaranteeing that what is tested in staging is exactly what runs in production.

Environment	Deployment Trigger	Purpose & Configuration
Development	Auto-deploy on every successful merge to 'develop' branch	Continuous integration environment for the team; used for early smoke testing and demoing in-progress features; uses synthetic/anonymized road-image data.
Testing / Staging	Auto-deploy of the same artifact promoted from Development after CI passes	Mirrors production configuration (same Kubernetes cluster topology, GIS integration, message-bus setup); automated

Environment	Deployment Trigger	Purpose & Configuration
		acceptance and load tests run here before promotion.
Production	Manual-approval-gated deployment using blue-green or canary rollout	Serves live municipal dashboards and public-safety alerts; canary releases expose a small percentage of GIS/dashboard traffic to the new version before full rollout, minimizing blast radius.

5.1 Blue-Green / Canary Rollout in Production

Production deployment uses a canary strategy: the new version is deployed alongside the current stable version, initially receiving a small slice of traffic (e.g., 10%) routed via the Kubernetes service mesh/ingress. Prometheus metrics (error rate, latency, AI-inference failure rate) are monitored for a defined bake period; if metrics remain healthy, traffic is gradually shifted to 100%. If metrics breach thresholds at any point, traffic is automatically routed back to the stable version and the canary is torn down.

6. Security, Quality Checks, Notifications & Rollback Mechanisms

6.1 Security Practices

- Secrets (API keys, database credentials, registry tokens) are stored in a secrets manager (e.g., Kubernetes Secrets sealed with SOPS, or a cloud secrets manager) and never committed to source control.
- SAST and dependency scanning run on every pipeline execution; container images are scanned before being pushed to the registry, and again before deployment to production.
- Role-based access control (RBAC) restricts who can approve production deployments and who can access production Kubernetes namespaces.
- All inter-service and external traffic is encrypted in transit (TLS); the pipeline verifies TLS certificate configuration as part of staging acceptance tests.

6.2 Quality Checks

- SonarQube quality gate enforces coverage, maintainability, and duplication thresholds before a build can be packaged.
- AI model validation tests block promotion if detection accuracy regresses beyond an agreed tolerance versus the currently deployed model.

6.3 Notifications

- Slack/Teams channels receive automated messages for: pipeline start, quality-gate/test failures (with a link to logs), successful staging deployment, production-approval requests, production deployment success, and any automated rollback event.
- Critical failures (e.g., production health-check breach) additionally trigger an on-call page via the alerting tool for immediate human attention.

6.4 Rollback Mechanisms

- Automated rollback: if post-deployment health checks or Prometheus SLA alerts fail during or shortly after a production rollout, Argo CD/Helm automatically reverts the release to the last known-good version — no manual intervention required for common failure modes.
- Manual rollback: release managers can trigger an instant rollback to any previously deployed, registry-stored image version through a single Helm/Argo CD command, supported by the immutable versioning applied at the packaging stage.
- Database/schema safety: migrations are designed to be backward-compatible for at least one release cycle, so a rollback of application code never leaves the database in an incompatible state.